KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH, Hyderabad

SKILLING-ML/DL

II-II CSE, CSE(AI&ML)

AY: 2025-2026

SL No	Week	Session-1 (3 Hours)	Detailed Subtopics	Session-2 (3 Hours)	Detailed Subtopics
1	Week 4	Unsupervised Learning & Model Issues	K-Means,Hierarchical Clustering, PCA	Overfitting and Underfitting	Regularization(L1/L2), Hyper parameter tuning

CONTENTS (Week 4-Session1 : Unsupervised Learning & Model Issues)

1. K-means clustering
2. Elbow method
3. Hierarchical clustering
4. Curse of Dimentionality
5. PCA theory and intuition
6. Eigenvalues and variance

1. What is Unsupervised Learning?

1.1 Unsupervised learning is a paradigm where we are given a dataset:

$$X = \{x_1, x_2, x_3, \ldots, x_n\}$$

with no labels.

The goal is to discover hidden structure in data.

Unlike supervised learning:

$$(xi, yi) \rightarrow \text{learn } f(x)$$

Unsupervised learning aims to:

- group similar data
- discover hidden patterns
- reduce dimensionality
- model data distribution

Key Difference from Supervised Learning

Aspect	Supervised	Unsupervised
Labels	Available	Not available
Goal	Predict outputs	Discover structure
Evaluation	Accuracy	Structure quality
Output	predictions	clusters, embeddings

1.2 Clustering Theory & Algorithms

Similarity & Distance Metrics

Clustering is based on a simple idea:

Similar data points should be closer to each other than to others.

To measure similarity, we define a **distance function**.

Let two data points be:

$$x = (x_1, x_2, \ldots, x_n)$$
$$y = (y_1, y_2, \ldots, y_n)$$

1.2.1 Euclidean Distance (Most Common)

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Geometric Meaning

Straight-line distance between two points.

Works well when:

- clusters are spherical
- features are continuous
- variance is similar in all directions

Example

Customer features:

- age
- income
- spending score

Euclidean distance measures overall similarity.

1.2.2 Manhattan Distance (L1 Distance)

$$d(x, y) = \sum | x_i - y_i |$$

Geometric Meaning

Distance along grid paths (like city blocks).

Works well when:

- features are independent
- high-dimensional data
- robust to outliers

Example

Used in recommender systems & high-dimensional text data.

1.2.3 Cosine Similarity (Angle-Based Similarity)

$$\text{similarity} = \frac{x \cdot y}{||x||\ ||y||}$$

Values:

- 1 → identical direction
- 0 → unrelated
- −1 → opposite

Works best for:

- text mining
- document similarity
- high-dimensional sparse vectors

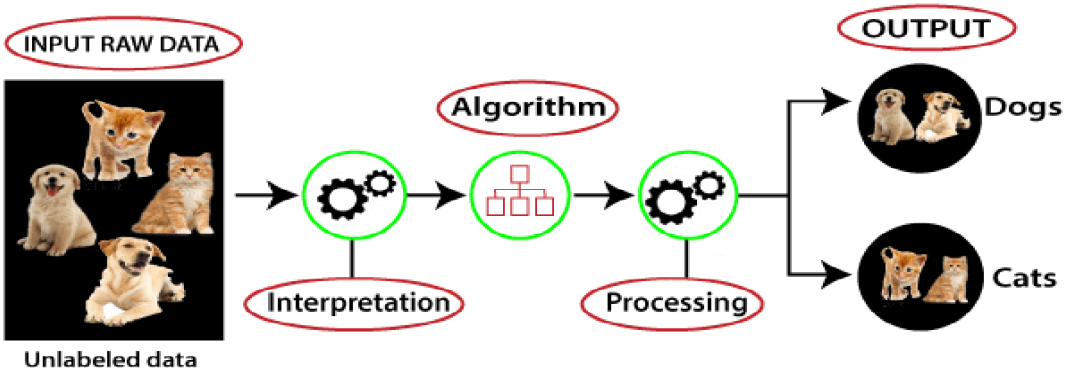
Example

Two documents with similar word patterns.

2. K-Means Clustering

2.1 What is Clustering?

- Unsupervised learning algorithms are used when we don't have labeled data. These models try to identify hidden structures or patterns in the data.
- The main objective is to explore the data and find inherent groupings or relationships.



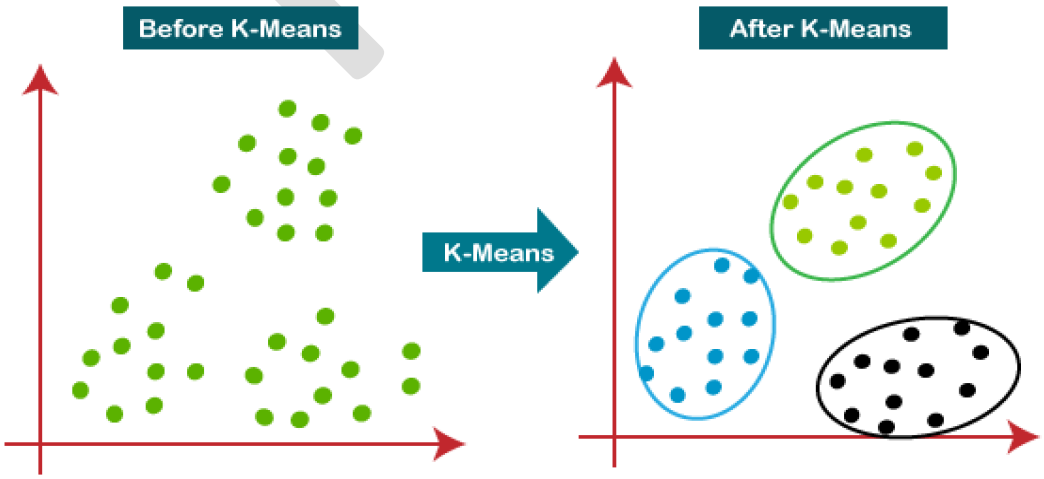
A) Supervised Learning: Like a student learning with a teacher providing answers and feedback.
B) Unsupervised Learning: Like a student discovering patterns and relationships on their own without answers provided.

2.2 Cluster Analysis

In unsupervised machine learning, cluster analysis groups unlabeled input data into meaningful output data. This output data can be used to understand the structure of the input data better or to make predictions about unlabeled data. There are many different ways to perform cluster analysis, but the most common method is to use a model that groups input data based on similarity.

2.3 K-means Clustering

Definition: K-Means is one of the most widely used clustering algorithms in unsupervised learning. It aims to partition a dataset into K clusters, where each data point belongs to the cluster with the nearest mean (centroid). The goal of K-Means is to minimize the variance within each cluster.



2.3.1 How does the K-Means Algorithm Work?

The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

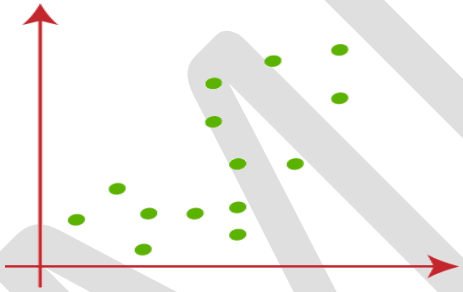
Step-4: Calculate the variance and place a new centroid of each cluster.

Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

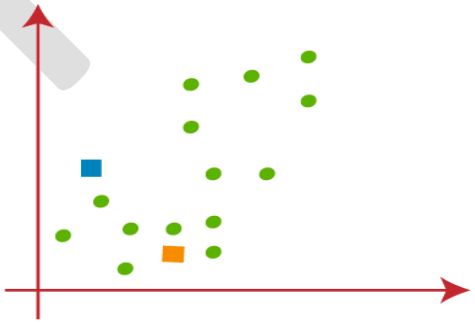
Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.

Suppose we have two variables M1 and M2. The x-y axis scatter plot of these two variables is given below:

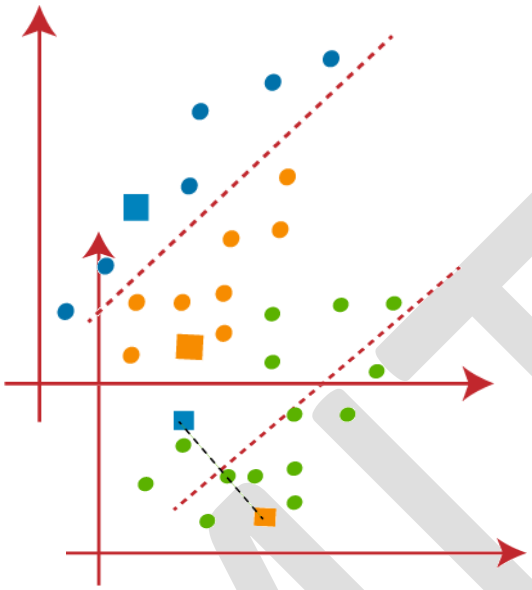


- Let's take number k of clusters, i.e., K=2, to identify the dataset and to put them into different clusters. It means here we will try to group these datasets into two different clusters.
- We need to choose some random k points or centroid to form the cluster. These points can be either the points from the dataset or any other point. So, here we are selecting the below two points as k points, which are not the part of our dataset. Consider the below image:



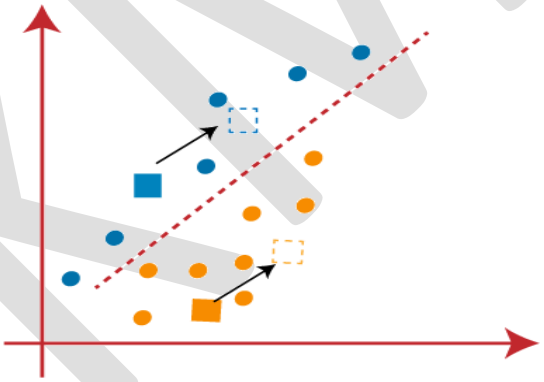
- Now we will assign each data point of the scatter plot to its closest K-point or centroid. We will compute it by applying some mathematics that we have studied to calculate the distance between two points. So, we will draw a median between both the centroids. Consider the below image:

From the above image, it is clear that points left side of the line is near to the K1 or blue centroid, and points to the right of the line are close to the yellow centroid. Let's color them as blue and yellow for clear visualization.

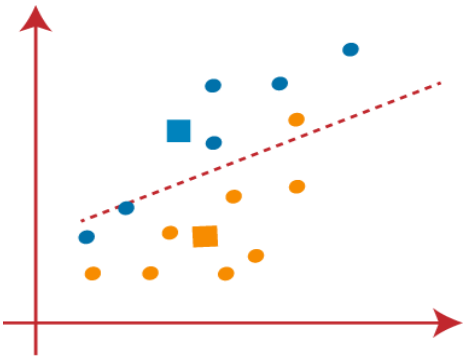


As we need to find the closest cluster, so we will repeat the process by choosing **a ne centroid**. To choose the new centroids, we will compute the center of gravity of

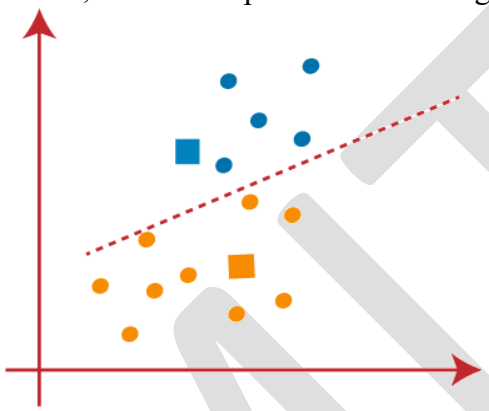
these centroids, and will find new centroids as below:



Next, we will reassign each datapoint to the new centroid. For this, we will repeat the same process of finding a median line. The median will be like below image:

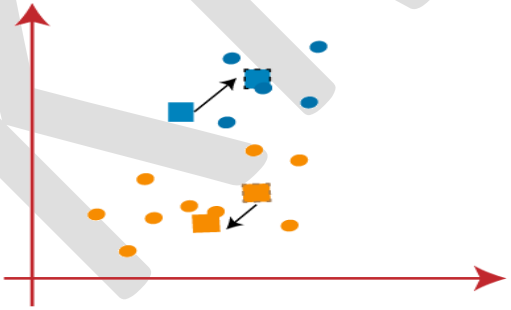


From the above image, we can see, one yellow point is on the left side of the line, and two blue points are right to the line. So, these three points will be assigned to new centroids.

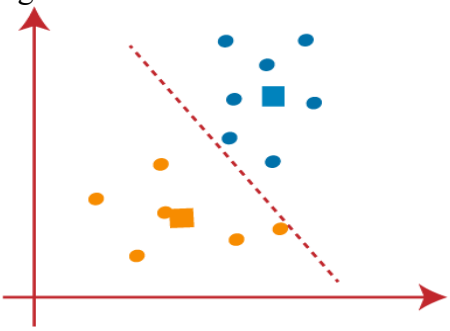


As reassignment has taken place, so we will again go to the step-4, which is finding new centroids or K-points.

- We will repeat the process by finding the center of gravity of centroids, so the new centroids will be as shown in the below image:

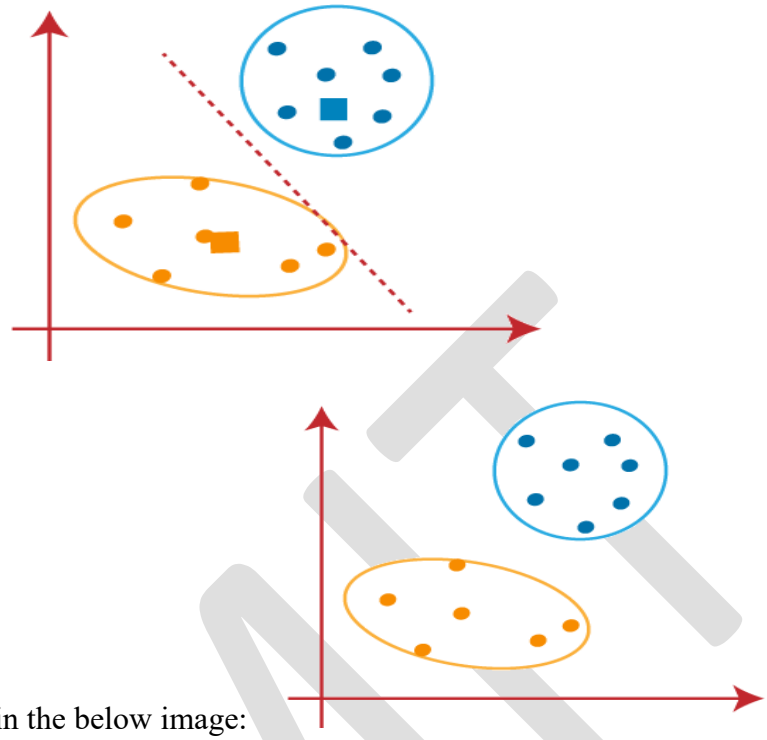


- As we got the new centroids so again will draw the median line and reassign the data points. So, the image will be:



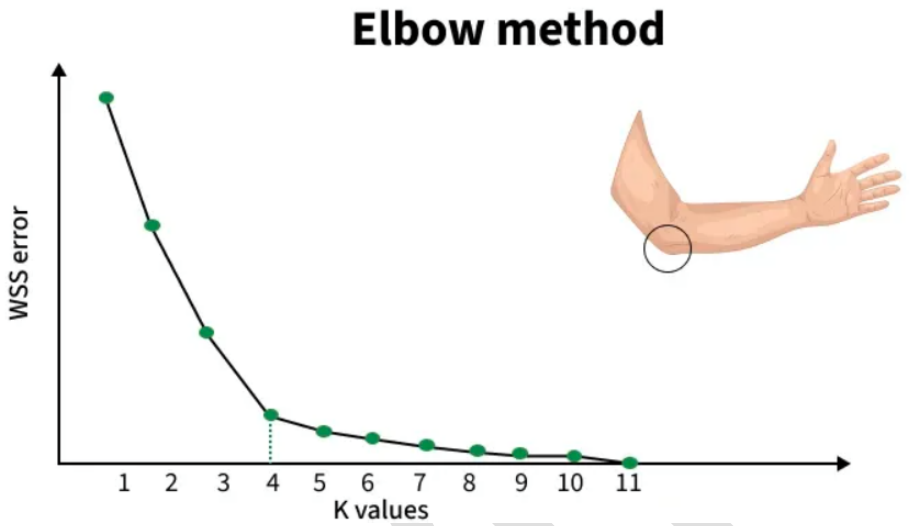
- We can see in the above image; there are no dissimilar data points on either side of the line, which means our model is formed. Consider the below image:

As our model is ready, so we can now remove the assumed centroids, and the two final clusters will be as shown in the below image:



3. Elbow Method

How to choose the value of "K number of clusters" in K-means Clustering?In K-Means clustering, the algorithm partitions data into k clusters by minimizing the distances between points and their cluster centroids. However, deciding the ideal k is not straightforward. The Elbow Method helps by plotting the Within-Cluster Sum of Squares (WCSS) against increasing k values and looking for a point where the improvement slows down, this point is called the "elbow."



Working of Elbow Point

The Elbow Method works in the following steps:

1. We begin by selecting a range of k values (for example, 1 to 10).

2. For each k, we run K-Means and calculate WCSS (Within-Cluster Sum of Squares), which shows how close the data points are to their cluster centroids:

$$\text{WCSS} = \sum_{i=1}^{k} \sum_{j=1}^{n_i} \text{distance}(x_j^{(i)}, c_i)^2$$

Where distance$(x_j^{(i)}, c_i)$represents the distance between the j^{th} data point $x_j^{(i)}$ in cluster i and the centroid $c_i$of that cluster.

3. After computing WCSS for all k values, we plot k vs WCSS.

4. WCSS always decreases as k increases because more clusters reduce the internal spread.

5. However, after a certain point, the improvement becomes very small. This bend or “elbow” in the curve indicates the point where adding more clusters no longer gives meaningful improvement.

Before the elbow: WCSS drops quickly -> clusters become much better.

After the elbow: WCSS drops slowly -> extra clusters add little value and may lead to overfitting.

The goal is to identify the point where the rate of decrease in WCSS sharply changes, indicating that adding more clusters (beyond this point) yields diminishing returns. This "elbow" point suggests the optimal number of clusters.

3.1 Limits of K-Means

While K-Means is a simple and effective algorithm, it has several limitations:

1. Predefined K:

Limitation: The number of clusters (K) must be specified beforehand. If you don't know the number of clusters in advance, this can be a challenge.

2. Sensitivity to Initial Centroids:

Limitation: K-Means is sensitive to the initial placement of centroids. If the initial centroids are poorly chosen, it can lead to poor clustering results.

3. Sensitivity to Outliers:

Limitation: K-Means is sensitive to outliers since they can significantly influence the centroid position.

4. Convergence to Local Minima:

Limitation: K-Means can converge to local minima, meaning it might not find the best possible cluster configuration. Running the algorithm multiple times with different initializations can help mitigate this

3.3 Solutions to Limitations:

- **Elbow Method**: To determine the optimal K by looking for a "knee" or "elbow" in the inertia plot (sum of squared distances of points to their centroids).
- **K-Means++**: A smarter initialization technique that spreads out the initial centroids, improving convergence and results.
- **Use of other algorithms**: When K-Means is unsuitable, alternative clustering algorithms like DBSCAN (density-based) or hierarchical clustering can be used.

4. Hierarchical Clustering

Hierarchical Clustering is an unsupervised learning method used to group similar data points into clusters based on their distance or similarity. Instead of choosing the number of clusters in advance, it builds a tree-like structure called a dendrogram that shows how clusters merge or split at different levels.

It helps identify natural groupings in data and is commonly used in pattern recognition, customer segmentation, gene analysis and image grouping.

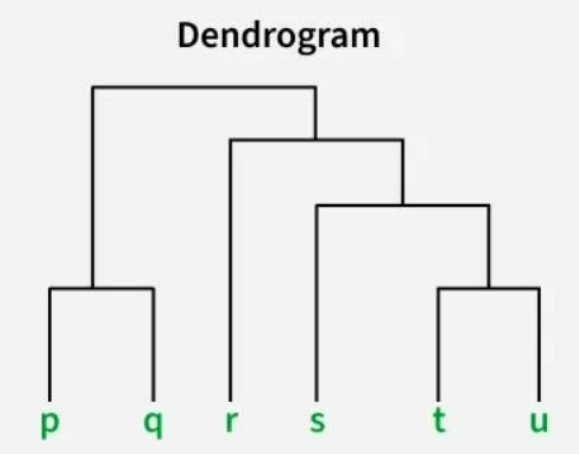
Imagine we have four fruits with different weights: an apple (100g), a banana (120g), a cherry (50g) and a grape (30g). Hierarchical clustering starts by treating each fruit as its own group.

- Start with each fruit as its own cluster.
- Merge the closest items: grape (30g) and cherry (50g) are grouped first.
- Next, apple (100g) and banana (120g) are grouped.
- Finally, these two clusters merge into one.

Finally all the fruits are merged into one large group, showing how hierarchical clustering progressively combines the most similar data points.

Dendrogram

A dendrogram is like a family tree for clusters. It shows how individual data points or groups of data merge together. The bottom shows each data point as its own group and as we move up, similar groups are combined. The lower the merge point, the more similar the groups are. It helps us see how things are grouped step by step.



- At the bottom of the dendrogram the points P, Q, R, S and T are all separate.
- As we move up, the closest points are merged into a single group.
- The lines connecting the points show how they are progressively merged based on similarity.
- The height at which they are connected shows how similar the points are to each other; the shorter the line the more similar they are.

5.Curse of Dimensionality

Curse of Dimensionality in Machine Learning arises when working with high-dimensional data, leading to increased computational complexity, overfitting, and spurious correlations.

Techniques like dimensionality reduction, feature selection, and careful model design are essential for mitigating its effects and improving algorithm performance. Navigating this challenge is crucial for unlocking the potential of high-dimensional datasets and ensuring robust machine-learning solutions.

Curse of Dimensionality refers to the phenomenon where the efficiency and effectiveness of algorithms deteriorate as the dimensionality of the data increases exponentially

5.1 How to Overcome the Curse of Dimensionality?

To overcome the curse of dimensionality, you can consider the following strategies:

5.1.1 Dimensionality Reduction Techniques:

- **Feature Selection:** Identify and select the most relevant features from the original dataset while discarding irrelevant or redundant ones. This reduces the dimensionality of the data, simplifying the model and improving its efficiency.
- **Feature Extraction:** Transform the original high-dimensional data into a lower-dimensional space by creating new features that capture the essential information. Techniques such as Principal Component Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t- SNE) are commonly used for feature extraction.

5.2.2 Data Preprocessing:

- **Normalization:** Scale the features to a similar range to prevent certain features from dominating others, especially in distance-based algorithms.
- **Handling Missing Values:** Address missing data appropriately through imputation or deletion to ensure robustness in the model training process.

6.PCA

6.1 Principal component Analysis

Principal component analysis (PCA) is a dimensionality reduction and machine learning method used to simplify a large data set into a smaller set while still maintaining significant patterns and trends.

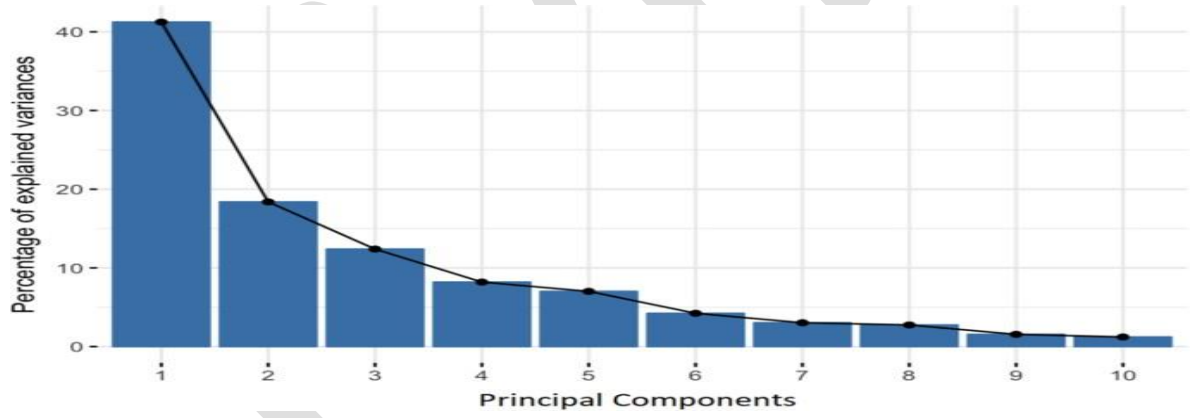
Reducing the number of variables of a data set naturally comes at the expense of

accuracy, but the trick in dimensionality reduction is to trade a little accuracy for simplicity. Because smaller data sets are easier to explore and visualize, and thus make analyzing data points much easier and faster for machine learning algorithms without extraneous variables to process.

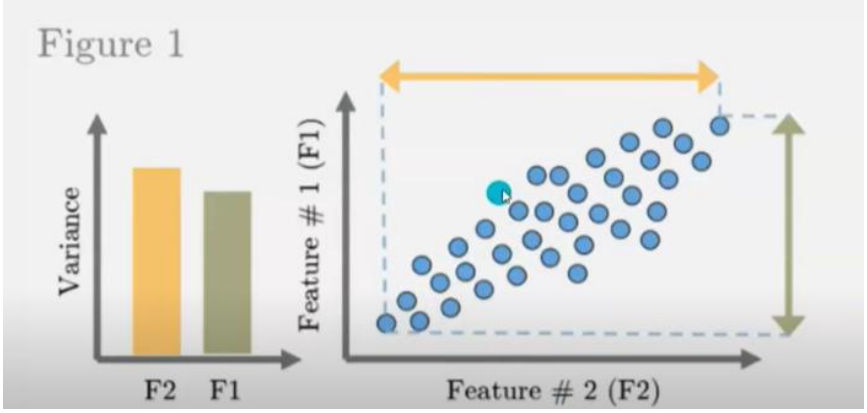
So, to sum up, the idea of PCA is simple: reduce the number of variables of a data set.

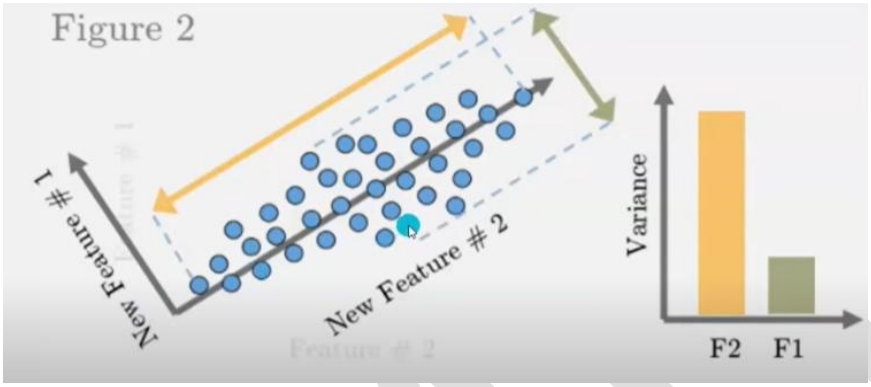
6.1.1 What Are Principal Components?

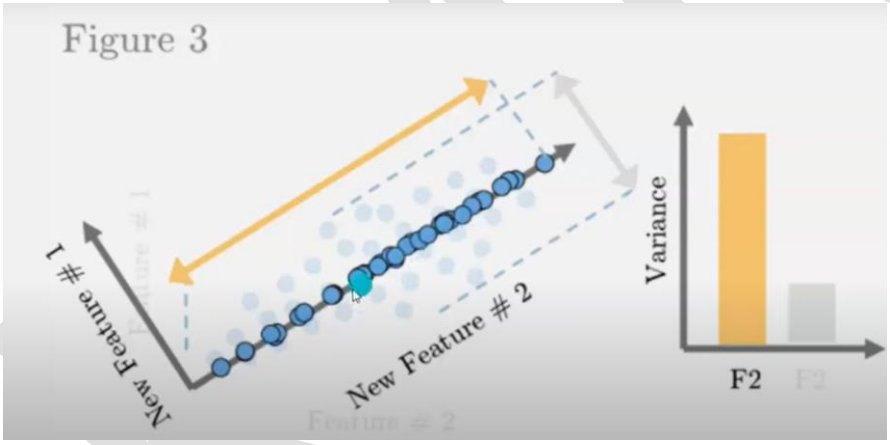
Principal components are new variables that are constructed as linear combinations or mixtures of the initial variables. These combinations are done in such a way that the new variables (i.e., principal components) are uncorrelated and most of the information within the initial variables is squeezed or compressed into the first components. So, the idea is 10-dimensional data gives you 10 principal components, but PCA tries to put maximum possible information in the first component, then maximum remaining information in the second and so on, until having something like shown in the scree plot below.



Geometrically speaking, principal components represent the directions of the data that explain a maximal amount of variance, that is to say, the lines that capture most information of the data. The relationship between variance and information here, is that, the larger the variance carried by a line, the larger the dispersion of the data points along it, and the larger the dispersion along a line, the more information it has. To put all this simply, just think of principal components as new axes that provide the best angle to see and evaluate the data, so that the differences between the observations are better visible.







6.1.2 How Do You Do a Principal Component Analysis?

1. Standardize the range of continuous initial variables
2. Compute the covariance matrix to identify correlations
3. Compute the eigenvectors and eigenvalues of the covariance matrix to identify the principal components
4. Create a feature vector to decide which principal components to keep
5. Recast the data along the principal components axes

a.Standardization of Data

PCA is sensitive to scale; therefore, the first step is to standardize each feature so that all

variables contribute equally.

That is, if there are large differences between the ranges of initial variables, those variables with larger ranges will dominate over those with small ranges (for example, a variable that ranges between 0 and 100 will dominate over a variable that ranges between 0 and 1), which will lead to biased results. So, transforming the data to comparable scales can prevent this problem.

$$z = \frac{x - \mu}{\sigma}$$

After standardization:

- Mean of each feature = 0
- Standard deviation = 1

This ensures that variables with larger numerical ranges do not dominate the analysis.

b. Compute the Covariance Matrix

The aim of this step is to understand how the variables of the input data set are varying from the mean with respect to each other, or in other words, to see if there is any relationship between them.

The covariance matrix is a p × p symmetric matrix (where p is the number of dimensions) that has as entries the covariances associated with all possible pairs of the initial variables. For example, for a 3-dimensional data set with 3 variables x, y,

$$\begin{bmatrix} Cov(x,x) & Cov(x,y) & Cov(x,z) \\ Cov(y,x) & Cov(y,y) & Cov(y,z) \\ Cov(z,x) & Cov(z,y) & Cov(z,z) \end{bmatrix}$$

and z, the covariance matrix is a 3×3 data matrix of this from:

Since the covariance of a variable with itself is its variance (Cov(a,a)=Var(a)), in the main diagonal (Top left to bottom right) we actually have the variances of each initial variable. And since the covariance is commutative (Cov(a,b)=Cov(b,a)), the entries of the covariance matrix are symmetric with respect to the main diagonal, which means that the upper and the lower triangular portions are equal.

c. Compute Eigenvalues and Eigenvectors

Eigenvectors and eigenvalues are the linear algebra concepts that we need to compute from the covariance matrix in order to determine the principal components of the

data.

What you first need to know about eigenvectors and eigenvalues is that they always come in pairs, so that every eigenvector has an eigenvalue. Also, their number is equal to the number of

dimensions of the data. For example, for a 3-dimensional data set, there are 3 variables, therefore there are 3 eigenvectors with 3 corresponding eigenvalues.

It is eigenvectors and eigenvalues who are behind all the magic of principal components because the eigenvectors of the Covariance matrix are actually the directions of the axes where

there is the most variance (most information) and that we call Principal Components. And eigenvalues are simply the coefficients attached to eigenvectors, which give the amount of variance carried in each Principal Component.

To identify directions of maximum variance, we compute eigenvalues and eigenvectors of the covariance matrix.

Eigenvalue equation:

$$\det(S - \lambda I) = 0$$

Solving this gives eigenvalues $(\lambda_1, \lambda_2, \ldots, \lambda_n)$.

Next, compute eigenvectors:

$$(S - \lambda I)v = 0$$

- Eigenvectors represent the directions of the new feature space (principal components).
- Eigenvalues indicate the amount of variance captured by each eigenvector.

The eigenvector with the highest eigenvalue becomes the first principal component (PC1).

d. Sort Eigenvalues in Descending Order

All eigenvalues are sorted from largest to smallest.

The principal components are ranked based on:

- PC1: highest variance
- PC2: second highest variance
- … and so on

This tells us which components carry the most information from the original dataset.

e. Select the Top k Principal Components

Depending on the dimensionality requirement, we select the top k eigenvectors.

For example:

- 10 features → reduce to 2 components → choose eigenvectors corresponding to the top 2 eigenvalues.

The selected eigenvectors form the projection matrix,

$$W = [v_1\ v_2\ \dots\ v_k]$$

f. Transform the Original Dataset

The final step projects the standardized dataset onto the new feature space:

$$X_{new} = X_{standardized} \cdot W$$

This produces the dataset in reduced dimensions.

After transformation:

- The features become uncorrelated
- Most variance is preserved
- The dataset becomes easier to visualize and process

6.1.3 Conclusion

PCA rotates the original feature space and selects new axes (principal components) that capture the maximum amount of variance. It helps simplify models, improves training speed, and removes redundancy in data while retaining essential information.

6.1.4 Limitations of PCA

1. PCA only captures linear relationships

PCA cannot detect non-linear patterns in data since it is based on covariance and linear algebra. This makes it unsuitable for datasets with curved or complex structures.

2. Principal components are difficult to interpret

The new transformed features (PC1, PC2, etc.) are combinations of original variables. They do not have direct physical meaning, making interpretation challenging in real-world applications.

3. Sensitivity to Outliers

- Outliers can heavily influence the principal components because PCA relies on variance maximization.
- Solution: Remove or handle outliers before applying PCA, or use robust PCA variants.

4.Loss of Information

- PCA reduces dimensionality by discarding less significant components, which might

result in the loss of important information.

- Trade-off: The balance between dimensionality reduction and retaining variance is subjective.

Numerical Example:

Dataset (5 samples × 2 features)

Let's use this simple dataset:

Sample	X_1	X_2
1	2.5	2.4
2	0.5	0.7
3	2.2	2.9
4	1.9	2.2
5	3.1	3.0

The goal: **Reduce 2D → 1D using PCA**.

STEP 1: Standardize / Mean-center the data

Compute means:

$$\bar{x}_1 = 2.04, \bar{x}_2 = 2.24$$

Subtract the mean:

Sample	X_1'	X_2'
1	0.46	0.16
2	-1.54	-1.54
3	0.16	0.66
4	-0.14	-0.04
5	1.06	0.76

STEP 2: Covariance Matrix

$$\Sigma = \begin{bmatrix} \text{cov}(X_1, X_1) & \text{cov}(X_1, X_2) \\ \text{cov}(X_2, X_1) & \text{cov}(X_2, X_2) \end{bmatrix}$$

Compute:

- $\text{Cov}(X_1, X_1) = \mathbf{0.6165}$
- $\text{Cov}(X_2, X_2) = \mathbf{0.716}$
- $\text{Cov}(X_1, X_2) = \mathbf{0.615}$

$$\Sigma = \begin{bmatrix} 0.6165 & 0.615 \\ 0.615 & 0.716 \end{bmatrix}$$

STEP 3: Eigenvalues & Eigenvectors

Solve the equation:

$$\det(\Sigma - \lambda I) = 0$$

Eigenvalues:

- $\lambda_1 = 1.284$
- $\lambda_2 = 0.049$

Eigenvectors:

$$v_1 = \begin{bmatrix} -0.6779 \\ -0.7352 \end{bmatrix}, v_2 = \begin{bmatrix} -0.7352 \\ 0.6779 \end{bmatrix}$$

The higher eigenvalue = **principal component direction**.

STEP 4: Choose Components

To reduce from 2D → 1D, take **PC1**:

$$W = v_1 = \begin{bmatrix} -0.6779 \\ -0.7352 \end{bmatrix}$$

STEP 5: Transform the Data

Use:

$$Z = X'W$$

We multiply each centered sample with PC1.

Sample	X_1'	X_2'	PC1 Score
1	0.46	0.16	-0.46
2	-1.54	-1.54	2.20
3	0.16	0.66	-0.77
4	-0.14	-0.04	0.15
5	1.06	0.76	-1.12

These **one-dimensional values represent the PCA output**.

Final PCA Result

The dataset:

- Originally **2D**
- After PCA → **1D**, capturing **most variance** in the data.

Final 1D transformed values:

[−0.46, 2.20, −0.77, 0.15, −1.12]

Q&A

1. An e-commerce company wants to group customers based on age, annual income, and spending score to run targeted marketing campaigns. Which algorithm would you choose and why?

Answer:

K-Means clustering is suitable because the data is numerical, and the goal is to divide customers into distinct groups based on similarity.

2. You applied K-Means with $k = 5$, but two clusters are very close and overlapping. What could be the reason?

Answer:

Poor choice of k, overlapping data distributions, or features not properly scaled.

3. A retail dataset has features like salary in lakhs and number of purchases. K-Means is giving incorrect clusters. What went wrong?

Answer:

Features are on different scales. K-Means is distance-based and requires feature normalization or standardization.

4. You are clustering images using K-Means, but clusters look meaningless. What could improve results?

Answer:

Apply PCA before K-Means to reduce dimensionality and noise.

5. A dataset contains many outliers. Is K-Means a good choice?

Answer:

No. K-Means is sensitive to outliers because centroids are based on mean values.

6. **How would you decide the optimal number of clusters in K-Means?**

   Answer:

   Using the Elbow method or Silhouette score..

7. In customer segmentation, K-Means creates clusters of very different sizes. Is this expected?

Answer:

K-Means assumes spherical and equal-sized clusters, so uneven cluster sizes indicate poor fit.

8. A medical dataset has 200 features, and the model is slow and overfitting. What would you apply?

Answer:

PCA to reduce dimensionality while preserving maximum variance.

9. Your PCA components are hard to interpret. Is this a drawback?

Answer:

Yes. PCA reduces interpretability because components are linear combinations of original features.

10. A dataset has strongly correlated features. How does PCA help?

Answer:

PCA converts correlated features into uncorrelated principal components.